

MITTUNIVERSITETET

Institutionen för data och elektroteknik (DET)

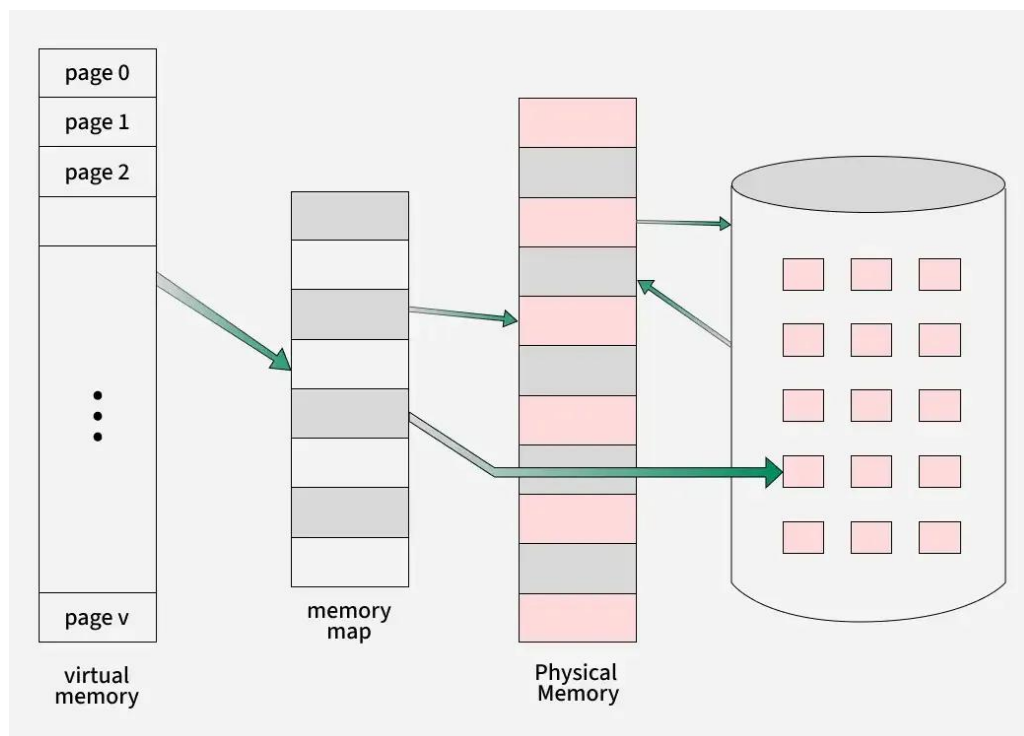
Examinator: Jimmy Åhlander, jimmy.ahlander@miun.se

Författarens e-postadress: gula2400@student.miun.se

Utbildningsprogram: Datateknik, 180hp

Omfattning: 1664 ord inklusive bilagor

Datum: 2025-12-14



Labb 3

Minnet – Operativsystem

9800x3d – 64GB 4800MT/s

Gunnar Landström

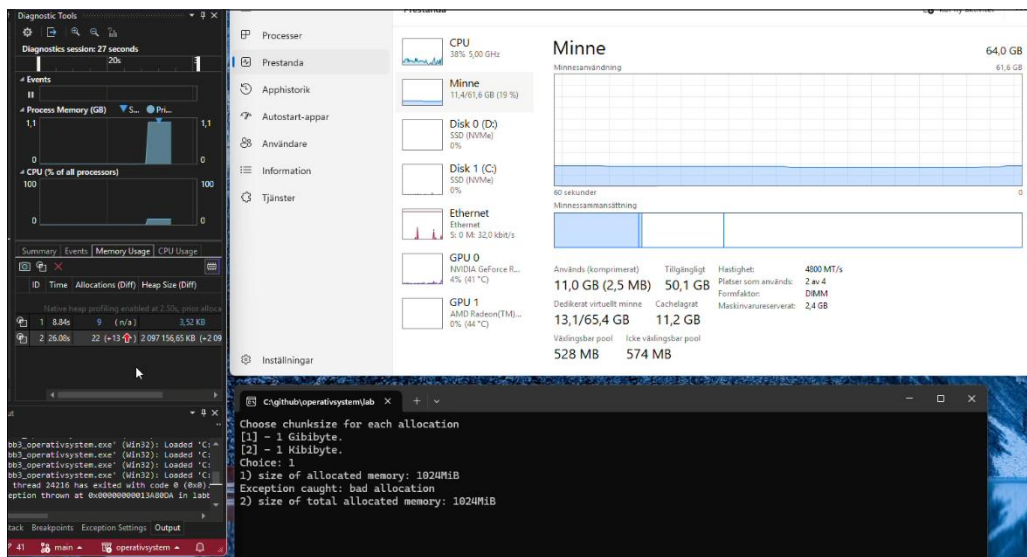
Innehåll

1	LAA OFF	4
1.1	64bit – GiB	4
1.1.1	Vilket fel när processen ej kan allokera mer minne?	4
1.1.2	Minnesmängd.	4
1.1.3	Tid. 0,077s	4
1.2	64bit - KiB	5
1.2.1	Vilket fel när processen ej kan allokera mer minne?	5
1.2.2	Minnesmängd.	5
1.2.3	Tid. 3,169s	5
1.3	32 bit – GiB	6
1.3.1	Vilket fel när processen ej kan allokera mer minne?	6
1.3.2	Minnesmängd.	6
1.3.3	Tid. 0,232s	6
1.4	32 bit – KiB	7
1.4.1	Vilket fel när processen ej kan allokera mer minne?	7
1.4.2	Minnesmängd.	7
1.4.3	Tid. 3,323s	7
2	LAA ON	8
2.1	32 bit – KiB	8
2.1.1	Vilket fel när processen ej kan allokera mer minne?	8
2.1.2	Minnesmängd.	8
2.1.3	Tid. 6,8s	8
2.2	32 bit – GiB	9
2.2.1	Vilket fel när processen inte kan allokera mer minne?	9
2.2.2	Minnesmängd.	9
2.2.3	Tid. 0,232s	9
2.3	64 bit – GiB	10
2.3.1	Vilket fel när processen inte kan allokera mer minne?	10
2.3.2	Minnesmängd.	10
2.3.3	Tid. 25,198s	10
2.4	64 bit – KiB	11
2.4.1	Vilket fel när processen inte kan allokera mer minne?	11
2.4.2	Minnesmängd.	12
2.4.3	Tid. 276s	12
3	Analys.....	13
3.1	Tabeller	13
3.1.1	LAA On	13

3.1.2	LAA Off	13
3.2	Diagram	14
3.3	Resultat	14
3.3.1	Disk	15
3.3.2	CPU	15
3.3.3	Cache	16
Källförteckning		17

1 LAA OFF

1.1 64bit – GiB



1.1.1 Vilket fel när processen ej kan allokeras mer minne?

När processen försöker allokeras mer minne än tillåtet så skickas ett exception om bad allocation.

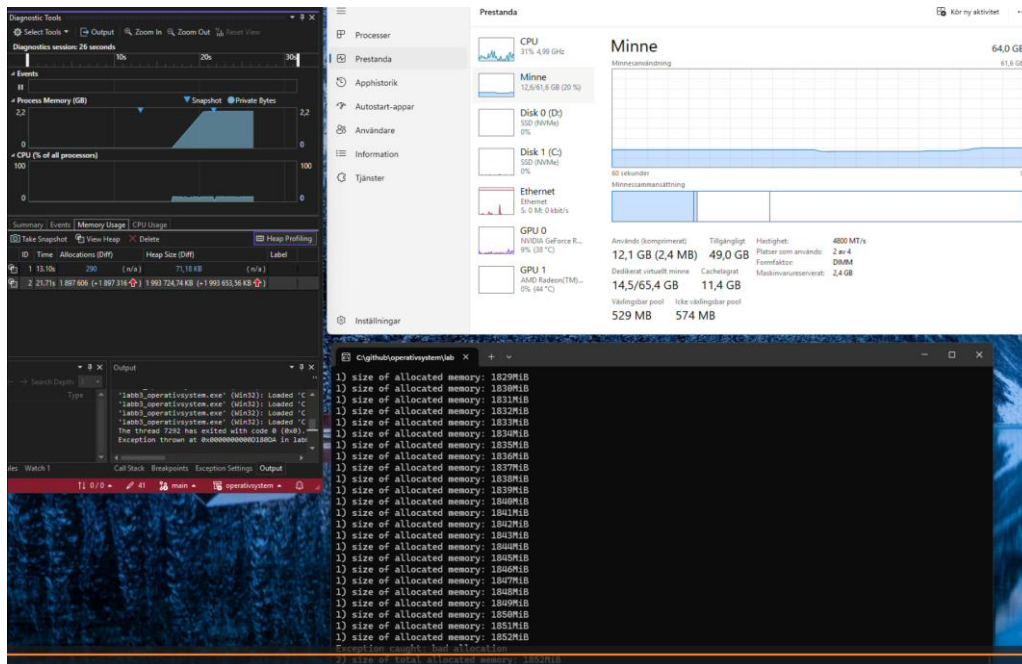
1.1.2 Minnesmängd.

Jämför man utskrifterna med Visual studio och aktivitetshanteraren ser man att alla matchar med en allokering på 1 GiB, men kollar man heapen i visual studio med hjälp av timestamps så är det ganska exakt 2 GiB som allokerats.

1.1.3 Tid. 0,077s

Körningen startas på 2MiB allokerat (Process Memory), och 0,077s senare så är 1 GiB allokerad

1.2 64bit - KiB



1.2.1 Vilket fel när processen ej kan allokerar mer minne?

När processen försöker allokerar mer minne än tillåtet så skickas ett exception om bad allocation.

1.2.2 Minnesmängd.

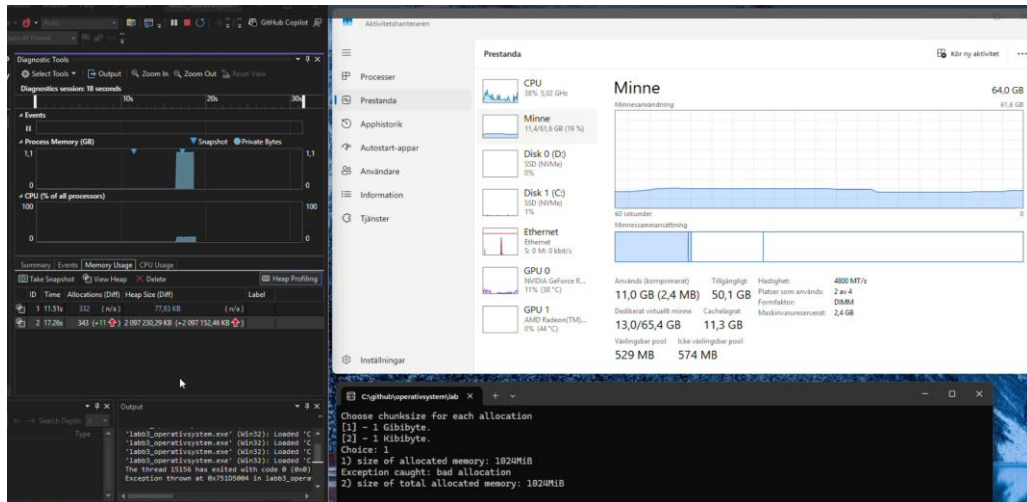
Denna gång ser vi i terminalen att vi får allokerar mer minne, 1852MiB och i aktivitetshanteraren ser vi att Windows använder runt 1GiB mer minne än tidigare test. Även Process Memory i Visual studio verkar rapportera ett tag på runt 2,2GiB.

Men tittar vi i stället på heapen igen, så är det faktiskt mindre minne som allokerats än föregående test och totalen landar på runt 1,9GiB

1.2.3 Tid. 3,169s

Körningen här startas även den på 2MiB enligt Visual Studio, sen tar det 3,169s innan den försöker allokerar otillåten minnesmängd och stannar då på 2GiB, något som överensstämmer med infon från heapen.

1.3 32 bit – GiB



1.3.1 Vilket fel när processen ej kan allokeras mer minne?

Även här stöter vi på ett bad allocation exception när vi försöker allokeras minne utanför vår tillåtna minnesrymd.

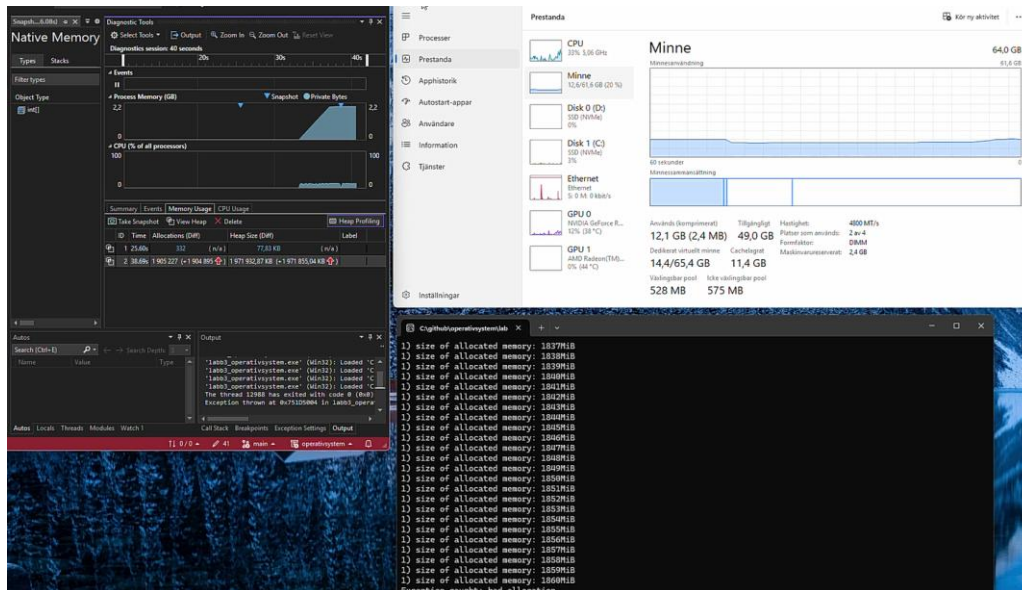
1.3.2 Minnesmängd.

Precis som i 64 bit så allokeras 1 GiB i terminalen, detta överensstämmer både med aktivitetshanteraren och Visual Studios graf Process Memory, men än en gång, när vi tar ett snapshot över vår heap så ser vi att det allokerats runt 2GiB

1.3.3 Tid. 0,232s

Här kollar process memory i Visual studio och vi startar med en allokering på 920KiB och 0,232s senare så har vi allokerat 2GiB

1.4 32 bit – KiB



1.4.1 Vilket fel när processen ej kan allokerar mer minne?

Vi försöker allokerar minne utanför processens minnesrymd så vi får ännu en bad allocation.

1.4.2 Minnesmängd.

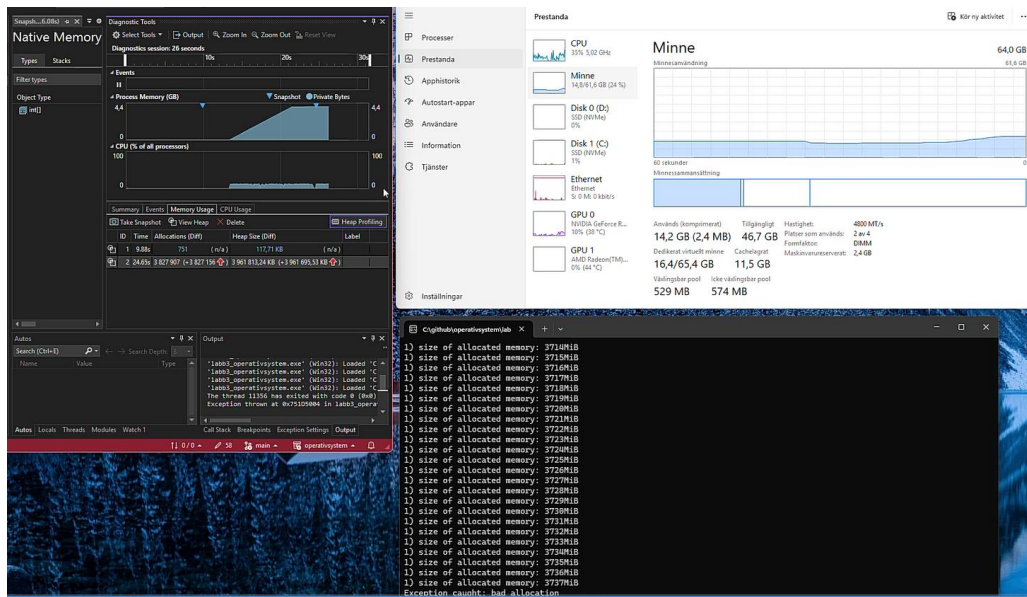
Precis som i 64 bitars så ser vi i terminalen att vi får allokerar mer minne, runt 1,85GiB, i aktivitetshanteraren ser vi samma mängd minne allokerat som i testet med 64 bit KiB, och i Visual Studios process memory så ser vi strax under 2GiB

1.4.3 Tid. 3,323s

I process memory så startas nu mätningen på 912KiB och 3,323s senare så stannar den på 2GiB

2 LAA ON

2.1 32 bit – KiB



2.1.1 Vilket fel när processen ej kan allokera mer minne?

Vi får en bad allocation när vi försöker allokera minne utanför processens minnesrymd.

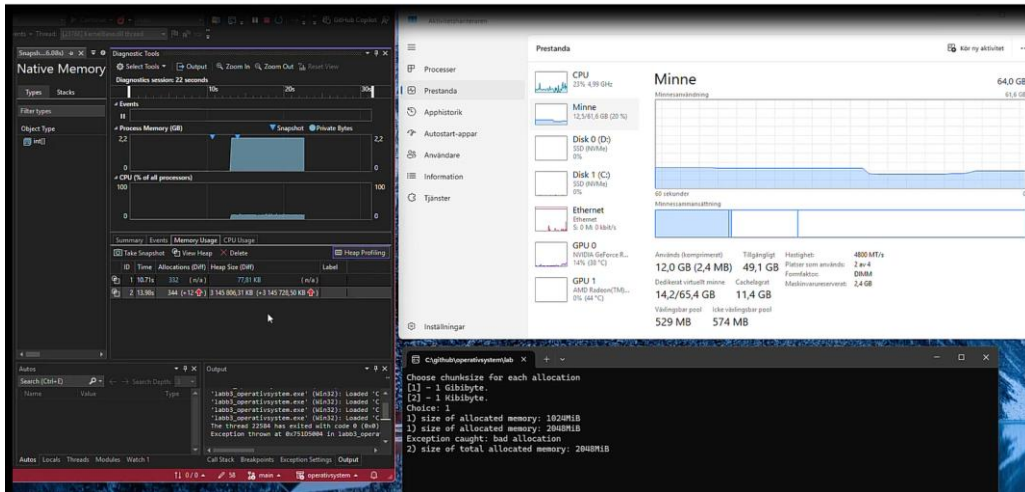
2.1.2 Minnesmängd.

I terminalen ser vi att vi får allokera 3737MiB vilket överensstämmer ganska bra med process memory och aktivitetshanteraren som har allokering på runt 4 GiB (Om vi tänker oss att basminnet är 10GiB)

2.1.3 Tid. 6,8s

Vi startar med en allokering på 924KiB i process memory och 6,8s senare så landar vi på 4GiB

2.2 32 bit – GiB



2.2.1 Vilket fel när processen inte kan allokerar mer minne?

Vi får ett bad allocation exception när vi försöker allokerar minne utanför vår tillåtna minnesrymd.

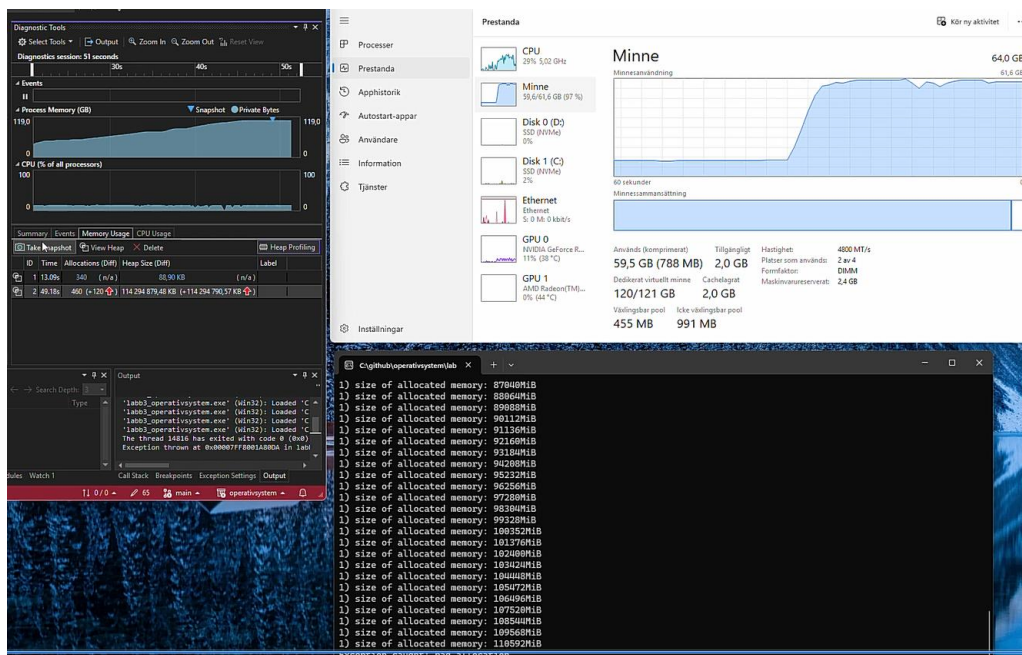
2.2.2 Minnesmängd.

I terminalen och aktivitetshanteraren ser vi att vi lyckas allokerar runt 2GiB och detta stämmer även överens med process memory i Visual Studio, men likt alla andra GiBtester så när vi snapshottar heapen ser vi en allokering på 1GiB mer, vilket ser konstigt ut.

2.2.3 Tid. 0,232s

Vi startar testet i Visual Studios process memory med en allokering på 920KiB och 0,232s senare landar vi på 2GiB.

2.3 64 bit – GiB

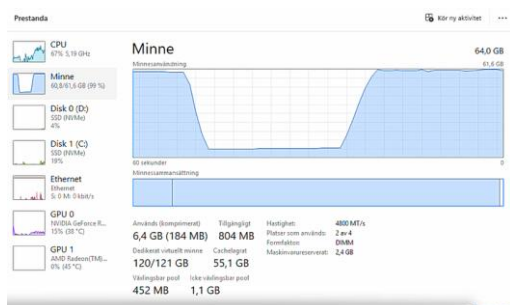


2.3.1 Vilket fel när processen inte kan allokerar mer minne?

Precis som föregående sex tester så får vi här ett bad allocation exception.

2.3.2 Minnesmängd.

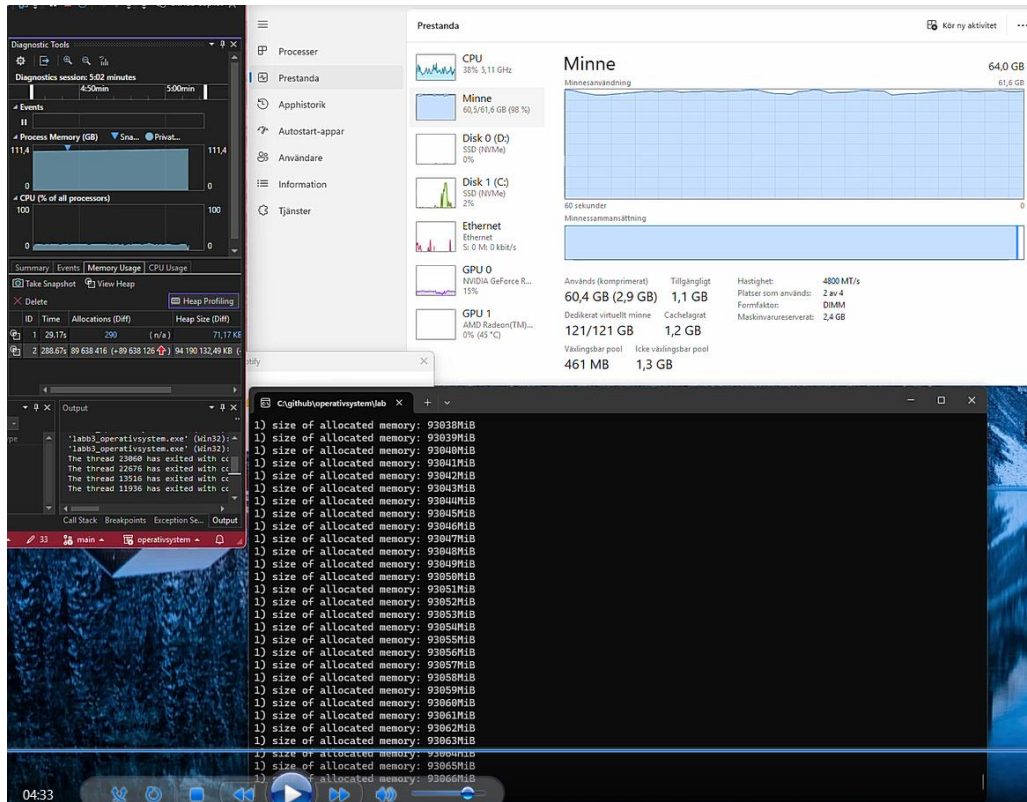
I terminalen, aktivitetshanteraren och Visual Studios process memory ser vi att vi landar på 110GiB allokerat minne. Men vi hittar också en anomali där vi i två separata tester ser att cachén bombas med lagring ena gången men inte den andra.



2.3.3 Tid. 25,198s

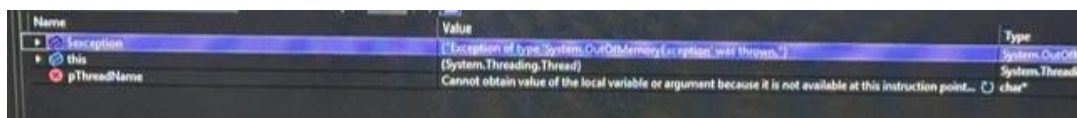
Vi startar mätningen i Visual Studios process memory på 740KiB och 25,198s senare så avslutar vi på 107,2GiB

2.4 64 bit – KiB



2.4.1 Vilket fel när processen inte kan allokeras mer minne?

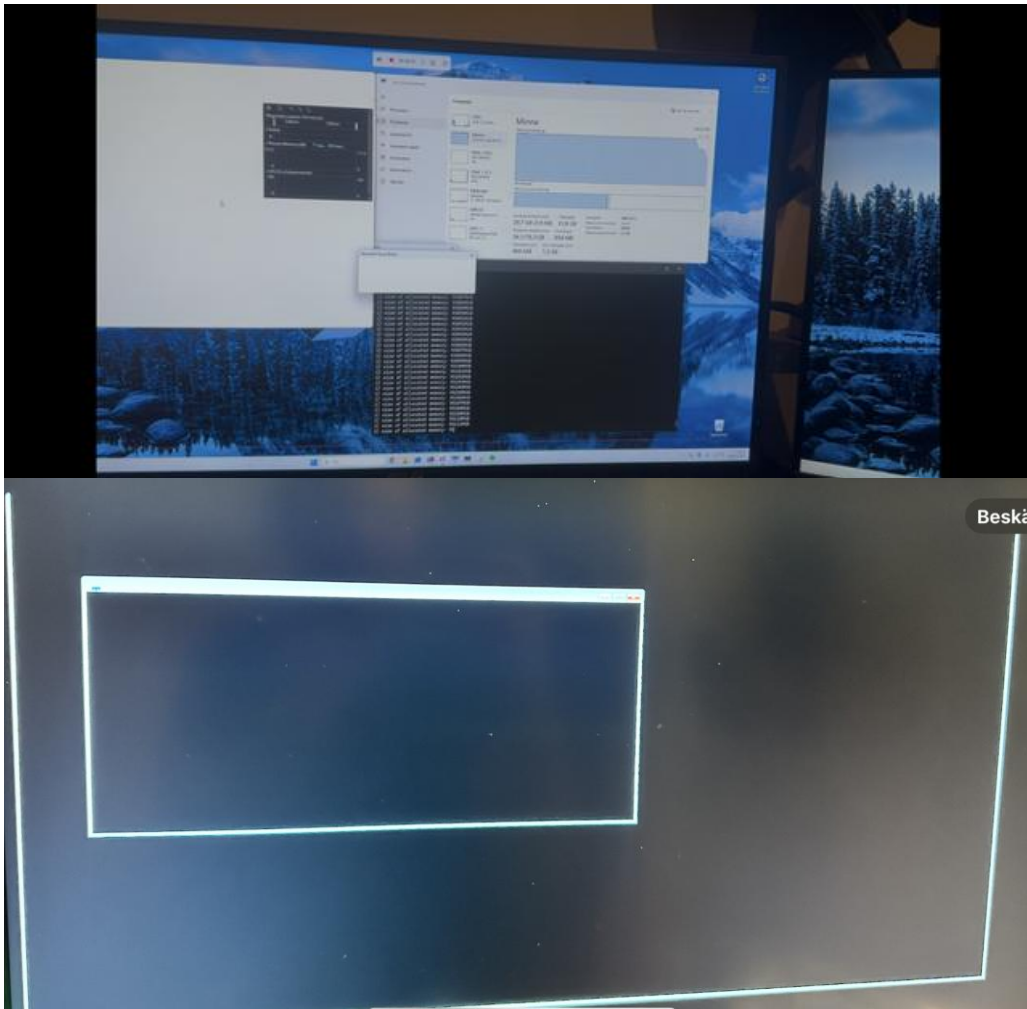
Det vanligaste resultatet är att datorn kraschar, men i några tester så gav den tillbaka ett läsvärt resultat.



System.OutOfMemoryException

Vi har alltså lyckats allokeras upp alla tillgängliga block av 1KiB som finns tillgängligt att det nu inte längre finns en plats att klämma in mer data, likt de tidigare GiB testerna utan FAA där vi tack vare minnesfragmenteringen inte får plats med vårt nya block av minne.

Annars har det pageats ut drivrutiner, och andra processer till den grad att allting fryser fast/eller kraschar.



2.4.2 Minnesmängd.

Vid detta test så allokerades det någonstans runt 93+GiB innan datorn började stöta på problem och den pagear ut processer, drivrutiner etc för att försöka göra plats för allokeringarna

2.4.3 Tid. 276s

Vi börjar med en allokering på 71,17KiB och avslutar med en allokering på ~93GiB+, då det inte fanns ett bra sätt att hinna få ett avslutningsvärde, men den närmsta snapshot av heapen jag lyckades få medan testet fortfarande kördes var ca 90GiB

3 Analys

3.1 Tabeller

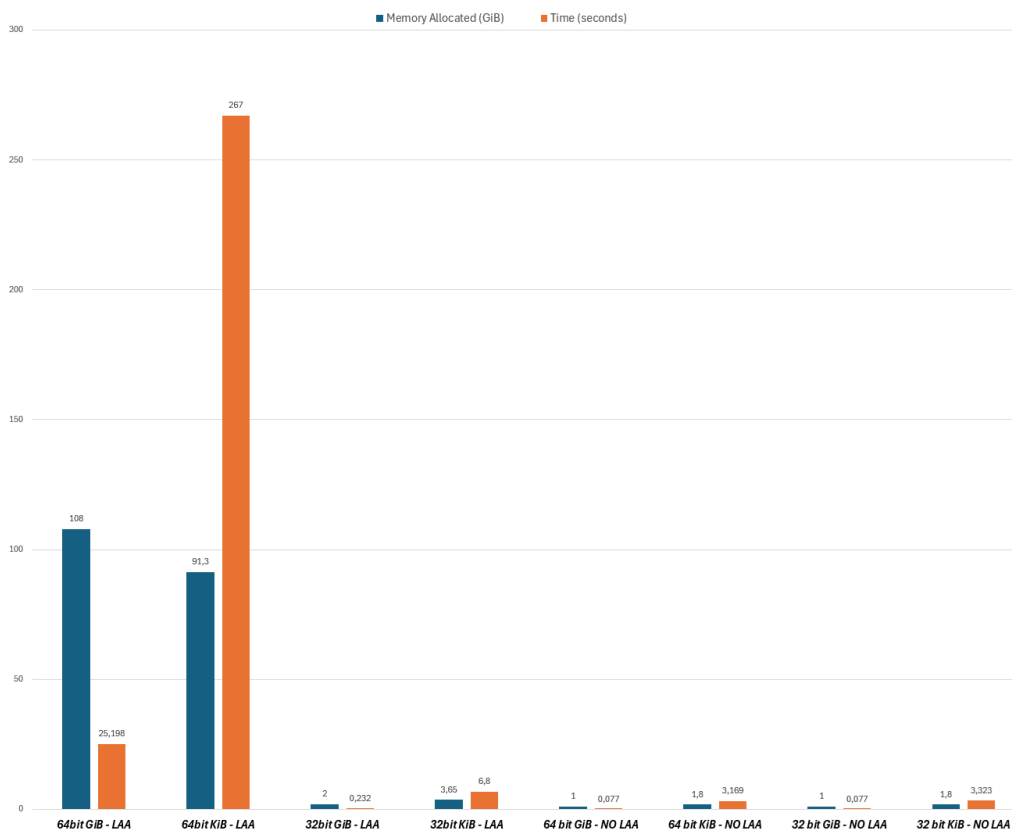
3.1.1 LAA On

	64bit GiB	64bit KiB	32bit GiB	32bit KiB
LAA	På	På	På	På
Minnesallokering	110 592 MiB	~93 000+ MiB	2048 MiB	3738 MiB
Tid	25,198s	276s	0,232s	6,8s

3.1.2 LAA Off

	64bit GiB	64bit KiB	32bit GiB	32bit KiB
LAA	Av	Av	Av	Av
Minnesallokering	1024 MiB	1852 MiB	1024 MiB	1860 MiB
Tid	0,077s	3,169s	0,077s	3,323s

3.2 Diagram



Ett diagram över hur stor heapen blivit under testerna och hur många sekunder det tagit.

3.3 Resultat

Vi ser att med Large Address Aware så ökar processens adressrymd från 2GB till 4GB på 32bits system medan på 64 bitars går vi från 2GB till gränser som stryps av hårdvara, där datorn som testerna gjorts på har 64GB fysiskt minne och får hjälp hårddisken att addera ytterligare virtuellt minne.

Så fort det virtuella minnet behöver expanderas för att öka den tillgängliga minnesrymden så vi kan pagea till hårddisken kostar det oerhörd mängd tid. Tittar vi till exempel på LAA on - 64bit GiB så tar det enbart 5,653s att allokeras 55,1GiB, men det tar i princip dubbelt så lång tid, 10,834s innan vi lyckats allokeras endast 5GiB till, efter det tar det nästan 15s till innan vi nått maxkapaciteten på 107,2GiB efter 25,198s.

Tittar vi på minneskompositionen i aktivitetshanteraren ser vi att det är det lediga minnet som allokeras först, sedan allokeras det allokeringbara minnet, till sist måste vi börja pagea ut datan till hårddisken i stället.

De praktiska gränserna sätts av Windows själva baserat på vilket operativsystem som används, i Windows 11 är det lägsta 128GB med Home versionen och 6TB för Pro/Enterprise versionen[1].

Tittar man på de teoretiska gränserna där man i 32bitars system når 4GiB, alltså $2^{32}B$, borde man i 64bitars system nå $2^{64}B$, alltså 16Exbi eller i tänkbar storlek 16 miljarder GiBi.

3.3.1 Disk

Tar man även en närmre titt på aktivitetshanterare och vad som händer med resten av datorn så kan man tydligt se ansträngningar när minnet börjar bli full och datorn försöker kompensera genom att allokera mer virtuellt minne genom att ta upp delar av hårddisken. Detta syns genom att diskanvändningen helt plötsligt ökar drastiskt när operativsystemet måste pagea data till disk för att kunna lösgöra någon plats att allokera mer minne på.



3.3.2 CPU

Kollar vi sedan på CPU-användningen när minnet har börjat bli fullt och datorn börjar allokera diskutrymme som virtuellt minne så får den jobba väldigt hårt för att försöka hinna med att allokera tillräckliga mängder samtidigt som den försöker allokera processens önskemål om att använda mer och mer minne, det uppstår alltså stora mängder sidfel och thrashing.



CPU-användning vid ett [LAA on 64 bit – GiB] test, över 80% på en AMD 9800X3D som klassats som världens snabbaste gaming CPU.

3.3.3 Cache

Två separata identiska tester, där ena testet använt sig av CPU:ns cache och det andra testet inte har gjort det, utan minnet registreras som att det används, men när processens allokeringar cachats så registreras en användning av 6,4GB. Bug?

Används (komprimerat)	Tillgängligt	Hastighet:	4800 MT/s
58,8 GB (788 MB)	2,7 GB	Platser som används:	2 av 4
Dedikerat virtuellt minne	Cachelagrat	Formfaktor:	DIMM
120/121 GB	2,7 GB	Maskinvarureserverat:	2,4 GB
Växlingsbar pool	Ikke växlingsbar pool		
455 MB	999 MB		

LAA on 64 bit – GiB | Ingen cache

Används (komprimerat)	Tillgängligt	Hastighet:	4800 MT/s
6,4 GB (184 MB)	804 MB	Platser som används:	2 av 4
Dedikerat virtuellt minne	Cachelagrat	Formfaktor:	DIMM
120/121 GB	55,1 GB	Maskinvarureserverat:	2,4 GB
Växlingsbar pool	Ikke växlingsbar pool		
452 MB	1,1 GB		

LAA on 64 bit – GiB | Huge cache

Källförteckning

- [1] Microsoft. Memory Limits for Windows and Windows Server Releases
[Internet] [uppdaterad 06/11/2025]
Hämtad från: <https://learn.microsoft.com/en-us/windows/win32/memory/memory-limits-for-windows-releases>

- [2] Wikipedia. Virtual memory [Internet] [uppdaterad 12/8/2025]
Hämtad från: https://en.wikipedia.org/wiki/Virtual_memory